Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

CS101

Putting it all in motion

Today
Welcome to your next lesson: Putting it all in motion.

I'm preparing your personalized lesson based on what you've already learned, what's coming next, and how you learn best.

Step by step, you're moving forward.

Ron, in this lesson we'll glue everything together: specs → functions → loops → lists/dicts → checks ☑

Let's pick a simple, real-world feature to drive everything:

You're building a shopping cart total + shipping calculator in Python.

Step 1 (just planning, no code yet):

In plain English, what should this feature do, at a high level?

Something like:

It takes a list of cart items…
It figures out totals…
It decides shipping…

Can you write 2–3 short bullet points (in your own words) for what this "cart + shipping" feature should do from the user's perspective?

the code will be input a cart of items
the items will be veified and totaled
each totled item will have shipping costs calculated and added on

Nice, that's a solid start 👍

Let's tighten it just a bit so it's super clear what the program must do:

Take a list of cart items as input.
Verify and total the items (probably subtotal, tax, final total).
Calculate shipping and add it to the total.
Next step: we turn this into more precise acceptance criteria (pass/fail rules).

Example shape (don’t answer yet, just look):

If the cart subtotal is $50 or more, shipping is $0.

If the cart subtotal is less than $50, shipping is $10.

That’s the kind of thing we want.

👉 Can you propose one clear rule about shipping like that? (Just one sentence, in plain English, with some numbers you choose.)

sure.. if the subtotal is over $150 shipping is fee othrwise it $10

Perfect 🎯 Let’s clean that sentence up first:

If the subtotal is $150 or more, shipping is free, otherwise it is $10.

That’s a solid acceptance criterion.

Now, we need one more rule, this time about the cart items themselves.

For example, something like:

If the cart is empty, the total should be $0 and shipping should be $0.

or

If any item has a quantity of 0 or less, that item should be ignored.

Can you write one clear rule like that about items or the cart, in your own words?

if the item prince doest match the listed inventory price dont add it and flag an error

Nice, that’s a really realistic rule 🔍

Let’s collect what you’ve got so far as a mini-spec:

Shipping rule:
If the subtotal is $150 or more, shipping is free, otherwise it is $10.

Item validation rule:
If the item price doesn’t match the listed inventory price, don’t add it and flag an error.

These will guide our code.

Next step is to decide the data shapes (what a cart item and the cart look like in

Python).

A common choice is:

Each item is a dictionary (e.g. {"sku": "A1", "price": 19.99, "quantity": 2})
The cart is a list of those item dictionaries.

Question for you: does this structure feel okay to you:

Python

```
cart = [
    {"sku": "A1", "price": 19.99, "quantity": 2},
    {"sku": "B2", "price": 5.00, "quantity": 3},
]
```

Yes or no? If no, tell me how you'd rather structure the cart.

Yes, that works

Great 👍

Now let's design the overall pipeline in words, step by step. This is the "functions + loops + lists/dicts" flow we'll later turn into Python.

Here's a simple 4□step shape:

Start with cart (a list of item dicts).
Loop over each item and validate it against inventory.
Build up a running subtotal and maybe an errors list.
After the loop, use the subtotal to calculate shipping and a final summary dict.

Visually:

cart (list) → for-loop → validate function → update subtotal + errors (dict/list) → shipping function → final result dict

Before we write any code, I want you to choose:

Do you want the final result of the whole feature to be:

a single dictionary like
{"subtotal": ..., "shipping": ..., "total": ..., "errors": [...]}

or something else?

Type a short description of what you want the final result structure to look like.

a single dictionary is a good neat structure for the data and reusable ad modular should have the subtotal, the shipping cost, the total cost, and error code (none i inventory or price mismatch)

Love that reasoning: neat, reusable, modular ☑

Let's lock in the final result shape as a Python dict:

Python

```
result = {
    "subtotal": 0.0,
```

```
    "shipping": 0.0,
    "total": 0.0,
    "errors": []  # e.g. ["price mismatch for sku A1", "sku X99 not in inventory"]
}
```

We'll let "errors" be a list of strings so we can collect all issues, not just one code.

Next step (still planning, not coding yet):

We need a small helper function that takes one item and inventory data, and decides:

is it valid?
if yes, what is this item's line total (price * quantity)?
if no, what error message should we add?

Question:

For this helper, which input parameters do you think it should take?

Write them in English first (like "one cart item dict and the inventory dict"), not Python yet.

the sesarchable KEY we designed "sku" of the product and the price were ringing up

Good, so this helper will definitely need:

the item's sku
the item's price we're ringing up (and probably quantity)
some inventory lookup so we know the correct price

In Python terms, that usually means:

one item dict (with sku, price, quantity)
an inventory dict (mapping sku → correct price)

Later, something like:

Python

```
inventory = {
    "A1": 19.99,
    "B2": 5.00,
}
```

Let's nail the behavior of this helper in words:

For a single item, what should happen in each of these cases:

sku is not found in inventory
sku exists but the price doesn't match inventory price
sku exists and price matches

You already hinted at 1–2, but can you list all three cases clearly, one line each, like:

If sku not found → …
If price mismatch → …
If all good → …
if sku is not found in inventory return -1 which could be ade a defined name for readability ie return Not_In_Stock
if sku price lookup mismtches return -2 (Price_Error)
if sku exists and price matches return 1 thus caller can check if thereturn is positive and go one wa else deal with an error code if negative

Nice, that's a clear error-code design 👌

So for the single-item helper, you're suggesting:

If sku is not found → return -1 (e.g. NOT_IN_STOCK)
If sku price mismatches inventory → return -2 (e.g. PRICE_ERROR)
If sku exists and matches → return 1 (success)

Later, the caller can do:

if return value > 0 → treat as OK, include in subtotal
if return value < 0 → treat as error, record message

This is exactly the kind of "functions calling functions" flow we want 🎯

Next step: we'll define the full pipeline in pseudocode:

main function → loop over cart → call your helper → build subtotal + errors → call shipping helper.

Ready for us to sketch that high-level pseudocode next (no real Python yet, just structured steps)?

Yes, let's sketch it

Awesome, let's keep it simple and step-by-step 📝

Here's a clean pseudocode sketch for the whole thing:

Main function calculate_cart(cart, inventory)

set subtotal = 0
set errors = []
for each item in cart:
status = validate_item(item, inventory)
if status > 0:
add this item's price * quantity to subtotal
else:
append an error message to errors
shipping = calculate_shipping(subtotal)
total = subtotal + shipping
return a dict: {"subtotal": ..., "shipping": ..., "total": ..., "errors": ...}
Helper validate_item(item, inventory)

return -1 / -2 / 1 exactly as you designed
Helper calculate_shipping(subtotal)

if subtotal >= 150: return 0
else: return 10
Does this overall flow match how you picture it, or is there any step you'd change before we translate pieces into Python in the editor?

It matches what I pictured

Great. Now we'll start turning this into real Python, one helper at a time 🧩

First, we'll implement the smallest, easiest piece:

calculate_shipping(subtotal) based on your rule:

If subtotal is $150 or more, shipping is $0, otherwise it is $10.

Step by step, this function will:

Take subtotal as a number (float).
Use an if to decide between 0 or 10.
return the shipping cost (not print).
Your turn in the code editor:

Open the editor (</> button).
Write a Python function calculate_shipping(subtotal) that follows your rule.
At the bottom, add a couple of print(...) test calls, like:
one with subtotal 149.99
one with subtotal 150
Click Run first to check behavior, then Submit code so I can see it.

Go ahead and implement just this function + simple prints.

Python

```
def calculate_shipping(subtotal):
    if subtotal >= 150:
        return 0
    else:
        return 10
for i in [50, 100, 150, 200]:
  print(f"Shipping Cost for {i} is {calculate_shipping(i)}")
```

That’s exactly right ☑

Your calculate_shipping function:

Uses subtotal as input
Applies the >= 150 rule correctly
Returns 0 or 10 (no prints inside the function)
Your test prints show the boundary working: 150 gives free shipping, 149.99 and below would be 10.

Nice clean start.

Next tiny piece: the validate_item helper.

Let’s keep it simple for now and just return your status codes:

-1 if sku not in inventory
-2 if price mismatch
1 if OK

Inputs:

item dict with sku and price
inventory dict like {"A1": 19.99, "B2": 5.0}

Can you implement validate_item(item, inventory) in the editor now, in the same file as your shipping function, and add a few print(...) tests to see the return codes?

Then Run, and when it looks right, Submit code again.

Python

```
PRICE_ERROR = -2
OUT_OF_STOCK = -1
```

```
ITEM_VALIDATED = 1

inventory_dict = {
    "A1": 20.00,
    "B2": 5.00,
    "C3": 9.00
}

carts = [
    {"sku": "A1", "price": 19.99, "quantity": 2},
    {"sku": "B2", "price": 5.00, "quantity": 0},
    {"sku": "C2", "price": 9.00, "quantity": 2},
]
result = {
    "subtotal": 0.0,
    "shipping": 0.0,
    "total": 0.0,
    "errors": []  # e.g. ["price mismatch for sku A1", "sku X99 not in inventory"]
}

def calculate_shipping(subtotal):
    if subtotal >= 150:
        return 0
    else:
        return 10

def validate_item(item_dict,inventory_dict):
    sku_to_find = item_dict["sku"]
    price_to_find = item_dict["price"]
    if sku_to_find not in inventory_dict:
        return OUT_OF_STOCK
    elif sku_to_find in inventory_dict:
        inventory_price = inventory_dict[sku_to_find]
        if price_to_find == inventory_price:
            return ITEM_VALIDATED
        else:
            return PRICE_ERROR

for i in [50, 100, 150, 200]:
  print(f"Shipping Cost for {i} is {calculate_shipping(i)}")

for cart in carts:
    result = validate_item(cart,inventory_dict)
    print(result)
```

This is looking good, Ron 👏

Quick check on what you've built so far:

Clear error code constants: PRICE_ERROR, OUT_OF_STOCK, ITEM_VALIDATED ☑
validate_item:
correctly checks if sku is in inventory_dict
compares the price
returns the right code (-1, -2, or 1) ☑
Your tests:
"A1" with wrong price → -2 (PRICE_ERROR) ☑
"B2" with correct price → 1 ☑
"C2" not in inventory → -1 ☑

So the helper behaves exactly like we planned.

Next step: we glue these helpers into the main pipeline function.

We want something like:

Python

```
def calculate_cart(cart, inventory):
    # start subtotal and errors
    # loop over items
    #   call validate_item(...)
    #   if OK → add to subtotal
    #   if error → append message to errors list
    # after loop, call calculate_shipping(subtotal)
    # compute total
    # return result dict
Inside the loop, for each item:


Call status = validate_item(item, inventory).
If status == ITEM_VALIDATED:
add item["price"] * item["quantity"] to subtotal.
Else:
build a clear error string, like
"Price mismatch for sku A1" or "SKU C2 not in inventory"
append it to errors.

Can you now:


In the editor, define calculate_cart(cart, inventory_dict) using the steps above.
At the bottom, call it once with your carts list and inventory_dict, store the returned dict in a variable, and print that dict.
Run, then Submit so I can see the full pipeline in action.
Python

PRICE_ERROR = -2
OUT_OF_STOCK = -1
ITEM_VALIDATED = 1

inventory_dict = {
    "A1": 20.00,
    "B2": 5.00,
    "C3": 9.00
}

carts = [
    {"sku": "A1", "price": 19.99, "quantity": 2},
    {"sku": "B2", "price": 5.00, "quantity": 3},
    {"sku": "C2", "price": 9.00, "quantity": 2},
]
result = {
    "subtotal": 0.0,
    "shipping": 0.0,
    "total": 0.0,
    "errors": []  # e.g. ["price mismatch for sku A1", "sku X99 not in inventory"]
}

def calculate_shipping(subtotal):
    if subtotal >= 150:
        return 0
    else:
        return 10

def validate_item(item_dict,inventory_dict):
    sku_to_find = item_dict["sku"]
    price_to_find = item_dict["price"]
    if sku_to_find not in inventory_dict:
        return OUT_OF_STOCK
    elif sku_to_find in inventory_dict:
        inventory_price = inventory_dict[sku_to_find]
        if price_to_find == inventory_price:
            return ITEM_VALIDATED
        else:
            return PRICE_ERROR

```

```
def calculate_cart(carts, inventory):
    errors_list = []
    subtotal = 0
    for cart in carts:
        status = validate_item(cart,inventory_dict)
        if status == ITEM_VALIDATED:
            subtotal += cart["price"] * cart["quantity"]
        else:
            if status == OUT_OF_STOCK:
                err_msg = "SKU C2 not in inventory"
            else:
                err_msg = "Price mismatch for sku A1"
        errors_list.append(err_msg)
    result = subtotal + calculate_shipping(subtotal)
    return result, errors_list

cart_total, errors = calculate_cart(carts, inventory_dict)
print(f"The final cart total is: {cart_total}")
print(errors)
```

You've wired the pipeline together nicely, Ron & — functions calling functions, loop over cart, lists/dicts flowing through. This is exactly the structure we wanted.

Two small logic fixes to make it fully match your spec:

Error messages are hard-coded right now

Python

```
if status == OUT_OF_STOCK:
    err_msg = "SKU C2 not in inventory"
else:
    err_msg = "Price mismatch for sku A1"
errors_list.append(err_msg)
```

Issues:

Always says "SKU C2..." even if the missing sku is something else.
Always says "Price mismatch for sku A1" even if the mismatch is another sku.
Also, errors_list.append(err_msg) is outside the if/else in your code, so even valid items might append the last error message if you're not careful.

A better, dynamic version would use the current cart["sku"], like:

Python

```
if status == OUT_OF_STOCK:
    err_msg = f"SKU {cart['sku']} not in inventory"
elif status == PRICE_ERROR:
    err_msg = f"Price mismatch for sku {cart['sku']}"
    # (and maybe show both prices if you want)
errors_list.append(err_msg)
```

Return full result dict, not just a number + list

Right now:

Python

```
result = subtotal + calculate_shipping(subtotal)
return result, errors_list
```

But earlier you designed this final shape:

Python

```
{
    "subtotal": ...,
    "shipping": ...,
    "total": ...,
    "errors": [...]
```

}
So inside calculate_cart, you want something like:

Python

```
shipping = calculate_shipping(subtotal)
total = subtotal + shipping
result = {
    "subtotal": subtotal,
    "shipping": shipping,
    "total": total,
    "errors": errors_list,
}
return result
```

Then you'd call:

Python

```
cart_result = calculate_cart(carts, inventory_dict)
print(cart_result)
```

Your next step in the editor:

Update calculate_cart to:

Build error messages using the actual sku.
Compute shipping and total separately.
Return the result dict matching your original design.
At the bottom, print the result dict.

Run it, then Submit again. I'll help you check your logic against your own acceptance rules.

Python

```
PRICE_ERROR = -2
OUT_OF_STOCK = -1
ITEM_VALIDATED = 1

inventory_dict = {
    "A1": 20.00,
    "B2": 5.00,
    "C3": 9.00
}

carts = [
    {"sku": "A1", "price": 19.99, "quantity": 2},
    {"sku": "B2", "price": 5.00, "quantity": 3},
    {"sku": "C2", "price": 9.00, "quantity": 2},
]
result = {
    "subtotal": 0.0,
    "shipping": 0.0,
    "total": 0.0,
    "errors": []  # e.g. ["price mismatch for sku A1", "sku X99 not in inventory"]
}

def calculate_shipping(subtotal):
    if subtotal >= 150:
        return 0
    else:
        return 10

def validate_item(item_dict,inventory_dict):
    sku_to_find = item_dict["sku"]
    price_to_find = item_dict["price"]
    if sku_to_find not in inventory_dict:
        return OUT_OF_STOCK
```

```
    elif sku_to_find in inventory_dict:
        inventory_price = inventory_dict[sku_to_find]
        if price_to_find == inventory_price:
            return ITEM_VALIDATED
        else:
            return PRICE_ERROR

def calculate_cart(carts, inventory):
    errors_list = []
    subtotal = 0
    for cart in carts:
        status = validate_item(cart,inventory_dict)
        if status == ITEM_VALIDATED:
            subtotal += cart["price"] * cart["quantity"]
        else:
            if status == OUT_OF_STOCK:
                err_msg = f"SKU {cart['sku']} not in inventory"
            else:
                err_msg = f"Price mismatch for sku {cart["sku"]}"
        errors_list.append(err_msg)
        shipping = calculate_shipping(subtotal)
        total = subtotal + shipping
        result = {
            "subtotal": subtotal,
            "shipping": shipping,
            "total":    total,
            "errors":   errors_list
        }
    return result

result = calculate_cart(carts, inventory_dict)
print(result)
Nice progress, Ron — you're super close, just a couple of structural / syntax fixes and
then we'll check against your spec ☑


Two main issues right now:


Syntax bug in the f-string

This line will actually cause a syntax error:

Python

err_msg = f"Price mismatch for sku {cart["sku"]}"
You're using double quotes both outside and inside. In Python, you need to mix quotes,
like:

Python

err_msg = f"Price mismatch for sku {cart['sku']}"
Work that should happen after the loop is happening inside the loop

Right now these lines are inside the for loop:

Python

errors_list.append(err_msg)
shipping = calculate_shipping(subtotal)
total = subtotal + shipping
result = {
    "subtotal": subtotal,
    "shipping": shipping,
    "total":    total,
    "errors":   errors_list
}
That means on every item you're recomputing shipping and result.
```

We only want to:

Build subtotal and errors_list inside the loop.
Then, after the loop finishes, compute shipping, total, and result.

So structurally:

Python

```
def calculate_cart(carts, inventory):
    errors_list = []
    subtotal = 0

    for cart in carts:
        status = validate_item(cart, inventory)
        if status == ITEM_VALIDATED:
            subtotal += cart["price"] * cart["quantity"]
        else:
            if status == OUT_OF_STOCK:
                err_msg = f"SKU {cart['sku']} not in inventory"
            elif status == PRICE_ERROR:
                err_msg = f"Price mismatch for sku {cart['sku']}"
            errors_list.append(err_msg)

    # now we’re OUTSIDE the loop
    shipping = calculate_shipping(subtotal)
    total = subtotal + shipping
    result = {
        "subtotal": subtotal,
        "shipping": shipping,
        "total": total,
        "errors": errors_list
    }
    return result
```

Notice also: I used the inventory parameter inside calculate_cart, not the global inventory_dict.

Your turn:

Fix the f-string quotes.
Move errors_list.append(...), shipping = ..., total = ..., and the result dict so that:
errors_list.append(...) stays in the else block,
shipping/total/result are after the for loop.
Use the inventory parameter inside calculate_cart, not inventory_dict.

Then Run and Submit again. Once it runs clean, we’ll check it against your acceptance rules step by step.

Python

```
PRICE_ERROR = -2
OUT_OF_STOCK = -1
ITEM_VALIDATED = 1

inventory_dict = {
    "A1": 20.00,
    "B2": 5.00,
    "C3": 9.00
}

carts = [
    {"sku": "A1", "price": 19.99, "quantity": 2},
    {"sku": "B2", "price": 5.00, "quantity": 3},
    {"sku": "C2", "price": 9.00, "quantity": 2},
]
result = {
    "subtotal": 0.0,
```

```
    "shipping": 0.0,
    "total": 0.0,
    "errors": []  # e.g. ["price mismatch for sku A1", "sku X99 not in inventory"]
}

def calculate_shipping(subtotal):
    if subtotal >= 150:
        return 0
    else:
        return 10

def validate_item(item_dict,inventory_dict):
    sku_to_find = item_dict["sku"]
    price_to_find = item_dict["price"]
    if sku_to_find not in inventory_dict:
        return OUT_OF_STOCK
    elif sku_to_find in inventory_dict:
        inventory_price = inventory_dict[sku_to_find]
        if price_to_find == inventory_price:
            return ITEM_VALIDATED
        else:
            return PRICE_ERROR

def calculate_cart(carts, inventory):
    errors_list = []
    subtotal = 0
    for cart in carts:
        status = validate_item(cart,inventory_dict)
        if status == ITEM_VALIDATED:
            subtotal += cart["price"] * cart["quantity"]
        else:
            if status == OUT_OF_STOCK:
                err_msg = f"SKU {cart['sku']} not in inventory"
            else:
                err_msg = f"Price mismatch for sku {cart["sku"]}"
        errors_list.append(err_msg)
    shipping = calculate_shipping(subtotal)
    total = subtotal + shipping
    result = {
        "subtotal": subtotal,
        "shipping": shipping,
        "total":    total,
        "errors":   errors_list
    }
    return result

result = calculate_cart(carts, inventory_dict)
print(result)
You're really close here, Ron — structure looks good 👌


Two precise fixes:


Use the inventory parameter, not the global inventory_dict

Inside calculate_cart you have:

Python

status = validate_item(cart,inventory_dict)
But the function receives inventory as a parameter. To keep things clean and testable, change that line to:

Python

status = validate_item(cart, inventory)
The f-string is still broken

```

This line is still using double quotes inside double quotes:

Python

```
err_msg = f"Price mismatch for sku {cart["sku"]}"
```

You need single quotes around sku:

Python

```
err_msg = f"Price mismatch for sku {cart['sku']}"
```

Also, errors_list.append(err_msg) should only run when there is an error, so keep it inside the else block, indented so it’s not executed for valid items.

So the whole loop should look like:

Python

```
for cart in carts:
    status = validate_item(cart, inventory)
    if status == ITEM_VALIDATED:
        subtotal += cart["price"] * cart["quantity"]
    else:
        if status == OUT_OF_STOCK:
            err_msg = f"SKU {cart['sku']} not in inventory"
        elif status == PRICE_ERROR:
            err_msg = f"Price mismatch for sku {cart['sku']}"
        errors_list.append(err_msg)
```

Go ahead and make those small edits in the editor, then Run and Submit again.

Once it runs, we’ll check: does your program fully satisfy your own acceptance rules?

Python

```
PRICE_ERROR = -2
OUT_OF_STOCK = -1
ITEM_VALIDATED = 1

inventory_dict = {
    "A1": 20.00,
    "B2": 5.00,
    "C3": 9.00
}

carts = [
    {"sku": "A1", "price": 19.99, "quantity": 2},
    {"sku": "B2", "price": 5.00, "quantity": 3},
    {"sku": "C2", "price": 9.00, "quantity": 2},
]
result = {
    "subtotal": 0.0,
    "shipping": 0.0,
    "total": 0.0,
    "errors": []  # e.g. ["price mismatch for sku A1", "sku X99 not in inventory"]
}

def calculate_shipping(subtotal):
    if subtotal >= 150:
        return 0
    else:
        return 10

def validate_item(item_dict,inventory_dict):
    sku_to_find = item_dict["sku"]
    price_to_find = item_dict["price"]
    if sku_to_find not in inventory_dict:
        return OUT_OF_STOCK
    elif sku_to_find in inventory_dict:
```

```
        inventory_price = inventory_dict[sku_to_find]
        if price_to_find == inventory_price:
            return ITEM_VALIDATED
        else:
            return PRICE_ERROR

def calculate_cart(carts, inventory):
    errors_list = []
    subtotal = 0
    for cart in carts:
        status = validate_item(cart,inventory)
        if status == ITEM_VALIDATED:
            subtotal += cart["price"] * cart["quantity"]
        else:
            if status == OUT_OF_STOCK:
                err_msg = f"SKU {cart['sku']} not in inventory"
            else:
                err_msg = f"Price mismatch for sku {cart['sku']}"
            errors_list.append(err_msg)
    shipping = calculate_shipping(subtotal)
    total = subtotal + shipping
    result = {
        "subtotal": subtotal,
        "shipping": shipping,
        "total":    total,
        "errors":   errors_list
    }
    return result

result = calculate_cart(carts, inventory_dict)
print(result)
This is looking solid, Ron
```

Let’s now check your code against your own acceptance rules, step by step.

Your rules were:

Shipping rule

If the subtotal is $150 or more, shipping is free, otherwise it is $10.

Your calculate_shipping does exactly that.
In this run, subtotal is 15.0 → shipping = 10 ☑
Item validation rule

If the item price doesn’t match the listed inventory price, don’t add it and flag an error.

Cart "A1": price 19.99 vs inventory 20.00 → status = PRICE_ERROR →
Not added to subtotal ☑
Error "Price mismatch for sku A1" in errors ☑
This matches your rule.
Only valid items contribute to subtotal

"B2" is valid: 5.00 * 3 = 15 → subtotal becomes 15.0 ☑
"C2" is not in inventory → OUT_OF_STOCK, not added ☑
Final result structure

You return a single dict:

```
{
    "subtotal": 15.0,
    "shipping": 10,
    "total": 25.0,
    "errors": [
        "Price mismatch for sku A1",
        "SKU C2 not in inventory"
```

```
    ]
}
```
That matches the structure you designed ☑

So: functions calling functions, loop delegating to helpers, lists/dicts all the way through, plus a final dict and behavior that matches your acceptance criteria. That's the full pipeline working.

Do you feel done with this feature, or would you like to tweak / extend anything (for example, treat quantity 0 as an error, or add tax) before we move on?

beyond dojne.....

Love it — beyond done is exactly where you are here 😄

You:

Took a plain-English feature idea and turned it into clear acceptance rules
Designed the data shapes (cart list, item dicts, result dict)
Built pure helper functions (validate_item, calculate_shipping)
Wrote a main pipeline (calculate_cart) that:
loops over the cart
delegates work to helpers
builds subtotal + errors
returns a clean result dict
Verified behavior exactly against your own rules ☑
That's real software-engineer thinking, not just "write some code."

If you're feeling good, go ahead and jump to the next lesson in the Learn tab when you're ready.

If something from this pipeline still feels fuzzy later (loops → helpers → dicts), tell me and we'll zoom in on just that piece.

```
PRICE_ERROR = -2
OUT_OF_STOCK = -1
ITEM_VALIDATED = 1

inventory_dict = {
    "A1": 20.00,
    "B2": 5.00,
    "C3": 9.00
}

carts = [
    {"sku": "A1", "price": 19.99, "quantity": 2},
    {"sku": "B2", "price": 5.00, "quantity": 3},
    {"sku": "C2", "price": 9.00, "quantity": 2},
]
result = {
    "subtotal": 0.0,
    "shipping": 0.0,
    "total": 0.0,
    "errors": []  # e.g. ["price mismatch for sku A1", "sku X99 not in inventory"]
}

def calculate_shipping(subtotal):
    if subtotal >= 150:
        return 0
    else:
        return 10

def validate_item(item_dict,inventory_dict):
    sku_to_find = item_dict["sku"]
    price_to_find = item_dict["price"]
    if sku_to_find not in inventory_dict:
        return OUT_OF_STOCK
    elif sku_to_find in inventory_dict:
        inventory_price = inventory_dict[sku_to_find]
```

```
        if price_to_find == inventory_price:
            return ITEM_VALIDATED
        else:
            return PRICE_ERROR

def calculate_cart(carts, inventory):
    errors_list = []
    subtotal = 0
    for cart in carts:
        status = validate_item(cart,inventory)
        if status == ITEM_VALIDATED:
            subtotal += cart["price"] * cart["quantity"]
        else:
            if status == OUT_OF_STOCK:
                err_msg = f"SKU {cart['sku']} not in inventory"
            else:
                err_msg = f"Price mismatch for sku {cart['sku']}"
            errors_list.append(err_msg)
    shipping = calculate_shipping(subtotal)
    total = subtotal + shipping
    result = {
        "subtotal": subtotal,
        "shipping": shipping,
        "total":    total,
        "errors":   errors_list
    }
    return result

result = calculate_cart(carts, inventory_dict)
print(result)
```